

Introducción a JavaScript

10 ejercicios prácticos — de los fundamentos a los primeros programas

DOCUMENTO DE REFERENCIA

Introducción a JavaScript · Apuntes

Estos ejercicios repasan, en orden creciente de dificultad, los conceptos del documento: sentencias, variables, tipos de datos, operadores, estructuras de control, funciones, objetos, arrays, DOM y eventos.

Fácil (1-3)

Fácil-Medio (4-7)

Medio (8-10)

1 Variables y tipos primitivos

FÁCIL

Declara cinco variables, una de cada tipo primitivo (number, string, boolean, undefined y un valor null), usando `let` o `const` según corresponda. Después, imprime por consola el resultado de `typeof` para cada una.

2 Concatenación y plantillas literales

FÁCIL

Crea dos variables `nombre` y `edad`. Construye la frase "Me llamo X y tengo Y años" de dos formas: usando el operador `+` y usando template literals (comillas invertidas).

3 Operadores aritméticos y de comparación

FÁCIL

Dadas dos variables `base` y `altura` de un rectángulo, calcula su área y su perímetro. Después, compara con `==` y con `===` el número `10` y el string `'10'`, e imprime ambos resultados explicando la diferencia.

4 Condicionales: if-else y switch

FÁCIL-MEDIO

Escribe una función que reciba una nota numérica (0-10) y devuelva "Suspenso", "Aprobado", "Notable" o "Sobresaliente" usando `if/else if`. Después, reescribe la misma lógica utilizando `switch` (pista: usa rangos con `true` como expresión del switch).

5 Operador ternario

FÁCIL-MEDIO

Reescribe el siguiente fragmento usando el operador ternario:

```
let mensaje;
if (edad >= 18) {
  mensaje = 'Eres mayor de edad';
} else {
  mensaje = 'Eres menor de edad';
}
```

6 Bucles for, while y do-while

MEDIO

Escribe una función `tablaDe(numero)` que imprima por consola la tabla de multiplicar de ese número del 1 al 10, usando un bucle `for`. Después, resuelve lo mismo con un `while` y con un `do-while`, y comenta cuándo tendría sentido usar cada uno.

7 Funciones y arrow functions

MEDIO

Crea una función `sumaArray(numeros)` que reciba un array y devuelva la suma de todos sus elementos usando un bucle. Reescribela después como arrow function.

8 Scope y hoisting

MEDIO

Analiza el siguiente código sin ejecutarlo y explica qué se imprime en cada `console.log`, justificando el motivo con lo aprendido sobre `var`, `let` y hoisting:

```
console.log(a);
var a = 5;

console.log(b);
let b = 10;
```

9 Objetos y arrays

MEDIO

Crea un objeto `libro` con las propiedades `titulo`, `autor` y `paginas`, y un método `resumen()` que devuelva un string con esa información. Después crea un array `biblioteca` con tres libros y recorre el array con un bucle `for` mostrando el resumen de cada uno.

10 DOM y eventos (bubbling)

MEDIO

Dado este HTML:

```
<div id="contenedor">
  <button id="miBoton">Haz clic</button>
</div>
```

Añade con JavaScript un `addEventListener('click', ...)` al botón y otro al div contenedor. Explica, basándote en el bubbling visto en el documento, en qué orden se ejecutarán ambos listeners al hacer clic en el botón, y cómo evitarías que el evento llegue al div usando `stopPropagation()`.

Soluciones orientativas

1. Usa `let x = 5; let y = 'texto'; let z = true; let w; let v = null;` y comprueba que `typeof w` es `'undefined'` y `typeof v` es `'object'` (peculiaridad histórica de JavaScript).
2. Con `+: 'Me llamo ' + nombre + ' y tengo ' + edad + ' años'`. Con template literals: ``Me llamo ${nombre} y tengo ${edad} años`` — más legible.
3. Área = base * altura; Perímetro = 2*(base+altura). `10 == '10'` es `true` (coerción de tipos); `10 === '10'` es `false` (compara también el tipo).
4. La lógica es la misma en ambos casos: agrupar la nota en cuatro tramos. En el switch, `switch(true) { case nota < 5: ...; case nota < 7: ...; }` simula rangos.
5. `let mensaje = edad >= 18 ? 'Eres mayor de edad' : 'Eres menor de edad';`
6. El resultado (tabla del 1 al 10) es el mismo con los tres bucles; `for` es ideal cuando se conoce el número de iteraciones, `while` cuando depende de una condición externa, y `do-while` cuando la primera ejecución debe garantizarse siempre.
7. `function sumaArray(numeros) { let total=0; for(const n of numeros) { total+=n; } return total; }` → arrow: `const sumaArray = (numeros) => numeros.reduce((a,b)=>a+b,0);`
8. El primer `console.log(a)` imprime `undefined` (var sufre hoisting e inicializa a undefined). El segundo lanza un `ReferenceError` porque `b` está en la temporal dead zone hasta su declaración con `let`.
9. El objeto define `titulo`, `autor`, `paginas` y un método que usa `this` para construir el string. El array se recorre con `for (let i=0;i.`
10. Al hacer clic en el botón, se ejecuta primero el listener del botón y después, por bubbling, el del div contenedor. Llamando a `event.stopPropagation()` dentro del listener del botón se evita que el evento suba al div.

